



Chuyên đề: Quy hoạch Động trên Cây (Tree Dynamic Programming)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

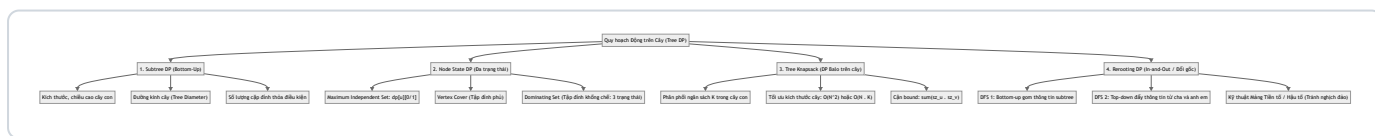
1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lập trình thi đấu (Competitive Programming), đồ thị dạng **Cây (Tree)** sở hữu hai đặc tính cấu trúc quan trọng nhất:

- Liên thông và không có chu trình (Acyclic):** Giữa hai đỉnh bất kỳ luôn tồn tại duy nhất một đường đi đơn.
- Cấu trúc đệ quy phân rã tự nhiên (Subtree Independence):** Khi cố định một đỉnh làm gốc (root), mỗi đỉnh con v của đỉnh u sẽ hình thành một cây con (subtree) độc lập. Mọi tương tác giữa các cây con của các anh em (siblings) đều buộc phải đi qua đỉnh cha chung u .

Nhờ hai đặc tính này, đồ thị cây là môi trường lý tưởng để áp dụng phương pháp **Quy hoạch động (Dynamic Programming)**:

- Không có chu trình đồng nghĩa đồ thị quan hệ phụ thuộc trạng thái luôn là một **DAG (Directed Acyclic Graph)**.
- Thứ tự duyệt **DFS hậu thứ tự (Post-order Traversal)** chính là một thứ tự sắp xếp Tô-pô (Topological Sort) tự nhiên: Ta giải quyết xong bài toán trên tất cả cây con của các nút con trước khi tổng hợp kết quả lên nút cha.



2. Kiến trúc Nền tảng: Mô hình Subtree DP (Bottom-Up)

2.1. Bản chất & Khung duyệt DFS

Với bài toán Subtree DP, thông tin tại đỉnh u phụ thuộc trực tiếp vào thông tin của các cây con gốc $v \in \text{children}(u)$.

Ta chọn một đỉnh bất kỳ làm gốc (thường chọn đỉnh 1) và gọi hàm `dfs(u, p)` với p là cha của u để tránh duyệt ngược lại cạnh vừa đi tới:

```

void dfs(int u, int p) {
    // 1. Khởi tạo giá trị cơ sở tại u (Base Cases)
    dp[u] = base_val;

    for (int v : adj[u]) {
        if (v == p) continue; // Tránh đi ngược lên cha
        dfs(v, u);           // Đệ quy giải quyết cây con gốc v trước

        // 2. Chuyển trạng thái: Cập nhật dp[u] từ dp[v]
        dp[u] = combine(dp[u], dp[v]);
    }

    // 3. Hoàn tất nút u (Post-visit finalize)
    dp[u] = finalize(dp[u]);
}

```

2.2. Bài toán Kinh điển: Đường kính Cây (Tree Diameter via DP)

Bài toán: Cho cây N đỉnh không trọng số (hoặc có trọng số dương). Tìm khoảng cách lớn nhất giữa hai đỉnh bất kỳ trên cây.

Mô hình hóa DP:

- Giả sử đường đi dài nhất trên cây có đỉnh cao nhất (Lowest Common Ancestor - LCA) là đỉnh u .
- Khi đó, đường kính sẽ đi qua u và tỏa xuống hai nhánh con sâu nhất của u .
- Gọi:
 - $h[u]$: Chiều cao lớn nhất từ đỉnh u xuống một lá bất kỳ thuộc cây con gốc u .
 - Khi xét lần lượt từng con $v \in \text{children}(u)$:
 - Đường đi dài nhất đi qua u nối hai nhánh con đã duyệt và nhánh con v là: $h[u] + (h[v] + 1)$.
 - Cập nhật đường kính toàn cục: $\text{diameter} \leftarrow \max(\text{diameter}, h[u] + h[v] + 1)$.
 - Cập nhật chiều cao của u : $h[u] \leftarrow \max(h[u], h[v] + 1)$.

```

int max_diameter = 0;
vector<int> h(N + 1, 0);

void dfs_diameter(int u, int p) {
    h[u] = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs_diameter(v, u);

        // Cập nhật đường kính đi qua u nối 2 nhánh con sâu nhất
        max_diameter = max(max_diameter, h[u] + h[v] + 1);
        // Cập nhật chiều cao cây con u
        h[u] = max(h[u], h[v] + 1);
    }
}

```

Mẹo / Tối ưu:

****So sánh với Thuật toán 2 lần BFS/DFS tìm đường kính**:**

- Thuật toán 2 lần BFS (bắt đầu từ đỉnh bất kỳ $\rightarrow$ tìm đỉnh x xa nhất $\rightarrow$ BFS từ x tìm đỉnh y xa nhất) chỉ áp dụng được cho **trọng số không âm**.
- Phương pháp Quy hoạch động trên cây tính được đường kính ngay cả khi cạnh có **trọng số thực âm hoặc dương**, đồng thời chỉ cần **1 lần duyệt DFS duy nhất** ($O(N)$).

3. Mô hình Trạng thái Đỉnh: Maximum Independent Set & Dominating Set

Khi việc chọn hoặc không chọn một đỉnh ảnh hưởng trực tiếp đến trạng thái hợp lệ của các đỉnh láng giềng kề cạnh, ta mở rộng mảng DP thêm chiều trạng thái quyết định của đỉnh u .

3.1. Tập Độc lập Lớn nhất trên Cây (Maximum Weight Independent Set)

Bài toán: Cho cây N đỉnh, mỗi đỉnh u có trọng số $w_u \geq 0$. Chọn một tập hợp các đỉnh sao cho không có hai đỉnh nào kề nhau và tổng trọng số là lớn nhất.

Thiết lập Trạng thái:

- $dp[u][0]$: Tổng trọng số lớn nhất trong cây con gốc u , với điều kiện **không chọn đỉnh u** .
- $dp[u][1]$: Tổng trọng số lớn nhất trong cây con gốc u , với điều kiện **có chọn đỉnh u** .

Công thức Chuyển trạng thái:

1. Nếu **không chọn u** ($dp[u][0]$): Các con v của u có thể được chọn hoặc không chọn tùy ý (ta chọn phương án tối ưu hơn giữa 2 trường hợp):

$$dp[u][0] = \sum_{v \in \text{children}(u)} \max(dp[v][0], dp[v][1])$$

2. Nếu **chọn u** ($dp[u][1]$): Do tính chất tập độc lập, tất cả các con v kề với u **bắt buộc không được chọn**:

$$dp[u][1] = w_u + \sum_{v \in \text{children}(u)} dp[v][0]$$

```
long long dp[MAXN][2];

void dfs_mis(int u, int p) {
    dp[u][0] = 0;
    dp[u][1] = weight[u];

    for (int v : adj[u]) {
        if (v == p) continue;
        dfs_mis(v, u);

        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}
```

Đáp án bài toán: $\max(dp[root][0], dp[root][1])$. Độ phức tạp: $O(N)$.

3.2. Tập Khống chế Cực tiểu (Minimum Dominating Set trên Cây)

Bài toán: Chọn số lượng đỉnh ít nhất sao cho mọi đỉnh trên cây hoặc thuộc tập chọn, hoặc kề với ít nhất một đỉnh thuộc tập chọn.

Để giải quyết bài toán này, 2 trạng thái là không đủ vì ta cần phân biệt giữa việc "đỉnh u chưa được ai khống chế" và "đỉnh u đã được con khống chế". Ta cần **3 trạng thái**:

- $dp[u][0]$: Đỉnh u **được chọn vào tập**. Khi đó con v có thể ở bất kỳ trạng thái nào:

$$dp[u][0] = 1 + \sum_{v \in \text{children}(u)} \min(dp[v][0], dp[v][1], dp[v][2])$$

- $dp[u][1]$: Đỉnh u **không được chọn, nhưng được khống chế bởi ít nhất một nút con v** .

- Mọi con v đều không bắt buộc phải chọn, nhưng **phải tồn tại ít nhất một con v^* có $dp[v^*][0]$** :

$$dp[u][1] = \sum_v \min(dp[v][0], dp[v][1]) + \Delta$$

trong đó $\Delta = 0$ nếu đã có ít nhất một con chọn $dp[v][0]$, ngược lại $\Delta = \min_v (dp[v][0] - dp[v][1])$.

- $dp[u][2]$: Đỉnh u **không được chọn và chưa được khống chế bởi bất kỳ con nào** (đang đợi cha của u khống chế):

$$dp[u][2] = \sum_{v \in \text{children}(u)} dp[v][1]$$

4. Kỹ thuật Tree Knapsack: DP Balo trên Cây & Tối ưu $O(N^2)$

4.1. Mô hình Bài toán

Xét bài toán: Chọn K đỉnh trên cây sao cho thỏa mãn ràng buộc quan hệ cha-con (ví dụ: muốn chọn v thì phải chọn u là cha của v) với tổng giá trị là lớn nhất.

Mô hình chuyển trạng thái tại nút u khi gộp dần từng nhánh con v :

Gọi $dp[u][i]$ là giá trị tối ưu khi chọn i đỉnh trong cây con gốc u .

Khi gộp thêm cây con v , đây là bài toán **gộp hai balo**:

$$dp_{\text{new}}[u][i + j] = \max(dp_{\text{new}}[u][i + j], dp_{\text{old}}[u][i] + dp[v][j])$$

4.2. Phân tích Độ phức tạp & Bỏ đề Bounded Subtree Size

Nhiều thí sinh lầm tưởng rằng thuật toán trên có độ phức tạp $O(N^3)$ hoặc $O(N \cdot K^2)$ nếu $K = O(N)$.

Tuy nhiên, nếu ta **chặn chặt cận trên của vòng lặp theo kích thước thực tế của từng cây con**:

```

vector<int> sz(N + 1, 0);
vector<vector<long long>> dp(N + 1);

void dfs_knapsack(int u, int p) {
    sz[u] = 1;
    dp[u] = {0, val[u]}; // dp[u][0] = 0, dp[u][1] = val[u] (chọn chính u)

    for (int v : adj[u]) {
        if (v == p) continue;
        dfs_knapsack(v, u);

        vector<long long> next_dp(sz[u] + sz[v] + 1, -INF);

        // Giới hạn vòng lặp theo kích thước thực tế của u và v
        for (int i = 1; i <= sz[u]; ++i) {
            for (int j = 0; j <= sz[v]; ++j) {
                next_dp[i + j] = max(next_dp[i + j], dp[u][i] + dp[v][j]);
            }
        }

        sz[u] += sz[v];
        dp[u] = move(next_dp);
    }
}

```

🚩 Quan trọng:

****Định lý Bỏ đề Cặp đỉnh (Pairwise Node Counting)**:**

Mỗi cặp đỉnh bất kỳ (x, y) trên cây có đúng **một** Lowest Common Ancestor (LCA) duy nhất.

Thao tác lồng nhau `for (int i = 1; i <= sz[u]) for (int j = 0; j <= sz[v])` thực chất ghép một đỉnh x thuộc nhánh con đã duyệt với một đỉnh y thuộc nhánh con v tại nút LCA chung của chúng là u .

Do mỗi cặp đỉnh (x, y) chỉ được ghép đôi **đúng 1 lần duy nhất** trong toàn bộ quá trình chạy thuật toán:

$$\sum_u \sum_{v \in \text{children}(u)} sz[u] \times sz[v] = \binom{N}{2} = \frac{N(N-1)}{2} = O(N^2)$$

Do đó, nếu chặn kích thước cây con chính xác, độ phức tạp của Tree Knapsack toàn cây là **chính xác $O(N^2)$** , không phải $O(N^3)$!

Nếu bài toán giới hạn ngân sách chọn tối đa K đỉnh, bằng kỹ thuật min-cap $\min(sz[u], K)$, độ phức tạp giảm xuống **$O(N \cdot K)$** !

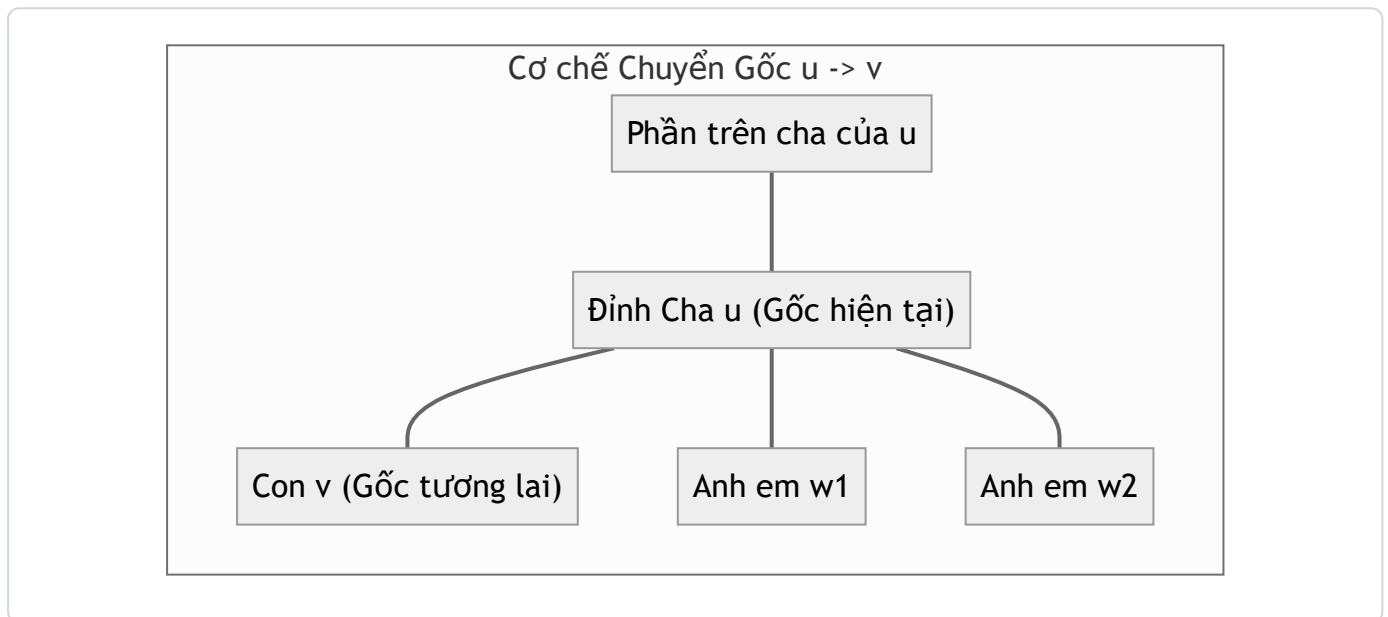
5. Kỹ thuật Đổi gốc (Rerooting DP / In-and-Out DP)

5.1. Động lực Bài toán

Trong nhiều bài toán cây, ta cần tính kết quả cho **tất cả** N **đỉnh** nếu lần lượt chọn mỗi đỉnh $r \in \{1, \dots, N\}$ làm gốc của cây:

- Nếu chạy DFS lại N lần: Độ phức tạp $O(N^2) \rightarrow$ TLE khi $N \geq 10^5$.
- Kỹ thuật **Rerooting DP** (hay **In-and-Out DP**) tính toán kết quả cho toàn bộ N đỉnh chỉ với **2 lượt duyệt DFS duy nhất** ($O(N)$):

1. **DFS 1 (Bottom-Up)**: Tính thông tin $in[u]$ chỉ trong phạm vi cây con gốc u (khi cố định gốc ban đầu là đỉnh 1).
2. **DFS 2 (Top-Down)**: Đẩy thông tin từ cha p xuống con v để bổ sung thông tin từ phần còn lại của cây nằm ngoài cây con v ($out[v]$).



5.2. Case Study 1: Tổng Khoảng cách tới Tất cả các Đỉnh (CSES Tree Distances II)

Bài toán: Cho cây N đỉnh. Với mỗi đỉnh u , tính tổng khoảng cách từ u tới toàn bộ $N - 1$ đỉnh còn lại:

$$ans[u] = \sum_{v=1}^N \text{dist}(u, v)$$

Phân tích Rerooting:

1. Lượt DFS 1 (Gốc là 1):

- $sz[u]$: Số lượng đỉnh trong cây con gốc u .
- $in[u]$: Tổng khoảng cách từ u tới tất cả các đỉnh trong cây con của u .
- Khi nối với con v : Mỗi đỉnh trong cây con v sẽ đi qua thêm cạnh (u, v) (khoảng cách tăng 1):

$$sz[u] = 1 + \sum_v sz[v], \quad in[u] = \sum_v (in[v] + sz[v])$$

1. Lượt DFS 2 (Chuyển gốc từ u sang con v):

2. Ban đầu tại gốc 1: $ans[1] = in[1]$.

3. Khi chuyển gốc từ u sang v (cạnh nối có độ dài 1):

- Mọi đỉnh thuộc cây con gốc v sẽ gần gốc mới v hơn 1 đơn vị $\implies$ tổng khoảng cách giảm $sz[v]$

- Tất cả các đỉnh còn lại ngoài cây con v ($N - sz[v]$ đỉnh) sẽ xa gốc mới v hơn 1 đơn vị $\implies$ tổng khoảng cách tăng $N - sz[v]$.

4. Công thức chuyển dịch biến thiên kỳ diệu:

$$ans[v] = ans[u] - sz[v] + (N - sz[v]) = ans[u] + N - 2 \cdot sz[v]$$

```
void dfs1(int u, int p) {
    sz[u] = 1;
    dp[u] = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        dp[u] += dp[v] + sz[v];
    }
}

void dfs2(int u, int p) {
    for (int v : adj[u]) {
        if (v == p) continue;
        // Chuyển gốc từ u sang v trong O(1)
        ans[v] = ans[u] + (N - sz[v]) - sz[v];
        dfs2(v, u);
    }
}
```

5.3. Case Study 2: Kỹ thuật Mảng Tiền tố & Hậu tố (Prefix/Suffix Aggregation)

Khi hàm kết hợp là các phép toán **không có phần tử nghịch đảo** (như $\max$, $\min$, $\gcd$, hoặc nhân Modulo 0), ta không thể lấy tổng của u trừ đi phần đóng góp của con v .

Giải pháp:

Tại mỗi nút u , ta liệt kê danh sách các con $v_1, v_2, \dots, v_k$.

Xây dựng 2 mảng dồn:

- `pref[i]`: Kết quả tích lũy từ con v_1 đến v_i .
- `suff[i]`: Kết quả tích lũy từ con v_i đến v_k .

Khi cần tính thông tin từ u truyền sang con thứ i (v_i), ta chỉ việc kết hợp:

$$\text{contrib} = \text{combine}(\text{pref}[i - 1], \text{suff}[i + 1], \text{from_parent})$$

Kỹ thuật này giữ vững độ phức tạp $O(N)$ tuyệt đối mà không cần bất kỳ phép trừ hay chia nghịch đảo nào!

6. Bảng Trace Chi tiết: Rerooting DP trên Cây 5 đỉnh

Xét cây gồm 5 đỉnh với các cạnh: $(1 - 2)$, $(1 - 3)$, $(2 - 4)$, $(2 - 5)$.



Bước 1: DFS 1 tính sz và in (gốc 1)

- Tại lá 3: $sz[3] = 1, in[3] = 0$
- Tại lá 4: $sz[4] = 1, in[4] = 0$
- Tại lá 5: $sz[5] = 1, in[5] = 0$
- Tại nút 2:
 - $sz[2] = 1 + sz[4] + sz[5] = 1 + 1 + 1 = 3$
 - $in[2] = (in[4] + sz[4]) + (in[5] + sz[5]) = (0 + 1) + (0 + 1) = 2$
- Tại gốc 1:
 - $sz[1] = 1 + sz[2] + sz[3] = 1 + 3 + 1 = 5$
 - $in[1] = (in[2] + sz[2]) + (in[3] + sz[3]) = (2 + 3) + (0 + 1) = 6$

Vậy với gốc 1: $ans[1] = 6$ (khoảng cách tới các đỉnh: $d(1, 1) = 0, d(1, 2) = 1, d(1, 3) = 1, d(1, 4) = 2, d(1, 5) = 2 \implies 0 + 1 + 1 + 2 + 2 = 6$).

Bước 2: DFS 2 Rerooting từ 1 sang các con

Áp dụng công thức: $ans[v] = ans[u] + N - 2 \cdot sz[v]$ với $N = 5$:

Bước chuyển gốc	Đỉnh cha u	Đỉnh con v	$sz[v]$	Công thức tính $ans[v]$	Giá trị $ans[v]$	Kiểm tra thực tế
Gốc ban đầu	-	1	-	$in[1]$	6	$0 + 1 + 1 + 2 + 2 = 6$
$1 \rightarrow 2$	1	2	3	$6 + 5 - 2 \times 3$	5	$d(2, \cdot) : 1 + 0 + 2 + 1 + 1 = 5$
$1 \rightarrow 3$	1	3	1	$6 + 5 - 2 \times 1$	9	$d(3, \cdot) : 1 + 2 + 0 + 3 + 3 = 9$
$2 \rightarrow 4$	2	4	1	$5 + 5 - 2 \times 1$	8	$d(4, \cdot) : 2 + 1 + 3 + 0 + 2 = 8$
$2 \rightarrow 5$	2	5	1	$5 + 5 - 2 \times 1$	8	$d(5, \cdot) : 2 + 1 + 3 + 2 + 0 = 8$

Mọi kết quả đều khớp chính xác tuyệt đối chỉ sau 2 lượt DFS!

! 7. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu

1. **Tràn Stack (Call Stack Overflow) với Cây suy biến dạng đường thẳng:**
2. Khi cây suy biến thành một chuỗi $N = 2 \cdot 10^5$ đỉnh, đệ quy DFS thông thường sẽ có độ sâu $2 \cdot 10^5$.
Trên Windows hoặc một số trình chấm giới hạn stack (như 8MB hay 16MB), chương trình sẽ bị **SIGSEGV / MLE / Runtime Error**.
3. **Khắc phục:**
 - Mở rộng stack trong g++ bằng cờ: `-w1, --stack, 268435456` (khi test trên máy).
 - Hoặc chuyển DFS sang khử đệ quy bằng cách dùng BFS tìm thứ tự Tô-pô ngược:

```
cpp // Mảng bfs lưu thứ tự duyệt; duyệt ngược từ cuối mảng là hậu thứ tự (Post-order) for
(int i = N - 1; i >= 0; --i) { int u = order[i]; // cập nhật dp lên cha parent[u] }
```
4. **Cập nhật Balo cây sai hướng dẫn đến lặp giá trị:**
5. Trong Tree Knapsack, nếu dùng mảng 1 chiều tại chỗ, các vòng lặp kích thước balo phải duyệt **ngược chiều** từ lớn về bé (giống 0-1 Knapsack) để tránh việc một nhánh con được dùng nhiều lần.
6. **Cấp phát động vector trong đệ quy gây quá thời gian (TLE):**
7. Tránh việc khởi tạo `vector<int> dp` mới bên trong thân hàm DFS nếu không cần thiết. Việc `malloc / free` 10^5 lần trong heap của hệ điều hành sẽ làm chậm chương trình gấp 5-10 lần. Ưu tiên cấp phát trước vùng nhớ cố định hoặc dùng `std::move`.
8. **Tràn số với phép cộng tích lũy:**
9. Khoảng cách trên cây với $N = 2 \cdot 10^5$ có thể lên tới $N \times N/2 \approx 2 \cdot 10^{10}$, vượt giới hạn kiểu `int` 32-bit ($2 \cdot 10^9$). Luôn dùng `long long` cho các mảng DP lưu khoảng cách và tích lũy.

📖 8. Tài liệu Tham khảo & Đọc thêm

1. **USACO Guide (Gold Section):**
2. [Tree DP: All Subtree Roots \(Rerooting DP\)](#). — Kỹ thuật đổi gốc 2 lượt DFS $O(N)$.
3. **VNOI Wiki:**
4. [Quy hoạch động trên cây \(Tree DP\)](#). — Subtree DP, đường kính cây và Tree Knapsack.
5. **CP-Algorithms:**
6. [Lowest Common Ancestor](#) — Tìm tổ tiên chung gần nhất và ứng dụng trên cây.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.